What is Verilog ?

Verilog is a hardware description language (HDL) that is used to describe digital systems and circuits in the form of code. It was developed by Gateway Design Automation in the mid-1980s and later acquired by Cadence Design Systems.

Verilog is widely used for design and verification of digital and mixed-signal systems, including both application-specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs). It supports a range of levels of abstraction, from structural to behavioral, and is used for both simulation-based design and synthesis-based design.

Discover more

Integrated circuit

Programming language

programming languages

The language is used to describe digital circuits hierarchically, starting with the most basic elements such as logic gates and flip-flops and building up to more complex functional blocks and systems. It also supports a range of modeling techniques, including gate-level, RTL-level, and behavioral-level modeling.

Real-World Application

Apple's M-series chips, Intel's Core processors, and AMD's Ryzen CPUs are all designed using Verilog and its successor SystemVerilog. Every modern smartphone, laptop, and data center relies on chips designed with HDLs like Verilog.

What was used before Verilog ?

Before the development of Verilog, the primary hardware description language (HDL) used for digital circuit design and verification was VHDL (VHSIC Hardware Description Language). VHDL was developed in the 1980s by the U.S. Department of Defense as part of the Very High-Speed Integrated Circuit (VHSIC) program to design and test high-speed digital circuits.

VHDL is a complex language that enables designers to describe digital systems using a range of abstraction levels, from the low-level transistor and gate levels up to complex hierarchical systems. It was designed to be more descriptive and flexible than earlier HDLs, such as ABEL (Advanced Boolean Expression Language). Here's an example of a AND gate described using VHDL.

Discover more

UVM register model

SoC verification platform

Debugging

```
-- and_gate.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Entity declaration: defines the inputs and outputs
```

```
entity and_gate is
 Port (
 A : in STD_LOGIC;
 B : in STD_LOGIC;
 Y : out STD_LOGIC
 );
end and_gate;

-- Architecture: describes the behavior or structure
architecture Behavioral of and_gate is
begin
 Y <= A and B;
end Behavioral;
```

.....

Code

```
-- and_gate.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Entity declaration: defines the inputs and outputs
entity and_gate is
 Port (
 A : in STD_LOGIC;
 B : in STD_LOGIC;
 Y : out STD_LOGIC
 );
end and_gate;

-- Architecture: describes the behavior or structure
architecture Behavioral of and_gate is
begin
 Y <= A and B;
end Behavioral;
```

Despite the development of Verilog and its increasing popularity since the 1980s, VHDL remains a widely used HDL, particularly in Europe and in the military and aerospace industries. Today, both Verilog and VHDL are widely used in digital circuit design and verification, with many companies and organizations using a combination of the two languages.

Why is Verilog better than its predecessor languages ?

Verilog introduced several important improvements over its predecessor languages, which helped make it a more popular and effective HDL for digital circuit design and verification. Here are a few reasons why Verilog is considered better than its predecessor HDLs:

- **Simpler syntax**: Verilog has a simpler syntax compared to VHDL, which allows designers to write code more quickly and with fewer errors.
- **Better support for behavioral modeling**: Verilog provides better support for describing the behavior and functionality of digital designs. It supports a range of modeling techniques,

from gate-level to behavioral-level modeling, which makes it easier to describe the behavior of complex digital circuits.

- **Higher level of abstraction**: Verilog provides a higher level of abstraction than its predecessor languages. It enables designers to describe digital circuits using concepts such as modules and ports, which makes the design process more efficient.
- **Better tool support**: Due to its increasing popularity, Verilog has better tool support than its predecessor languages. Verilog has a range of integrated development environments (IDEs) and simulation tools available, which makes it easier to design and verify digital circuits.

How is Verilog useful ?

Verilog creates a level of abstraction that helps hide away the details of its implementation and technology.

For example, the design of a D flip-flop would require the knowledge of how the transistors need to be arranged to achieve a positive-edge triggered FF and what the rise, fall and clk-Q times required to latch the value onto a flop among many other technology oriented details. Power dissipation, timing and the ability to drive nets and other flops would also require a more thorough understanding of the physical characteristics of a transistor.

Discover more

Combinational logic designs

Logic analyzer tool

Debugging power intent tools

Verilog helps us to focus on the behavior and leave the rest to be sorted out later.

Verilog Code Example

The following Verilog code describes the *behavior* of a counter. The counter counts up if the **up_down** signal is 1, and down if its value is 0. It also resets the counter if the signal **rstn** becomes 0, making it an active-low reset.

```
module ctr (input                    up_down,
                                     clk,
                                     rstn,
            output reg [2:0]   out);
      always @ (posedge clk)
            if (!rstn)
                  out <= 0;
            else begin
                  if (up_down)
                        out <= out + 1;
                  else
                        out <= out - 1;
            end
endmodule
```

The simple example shown above illustrates how all the physical implementation details (interconnection of underlying logic gates like NAND and NOR) have been hidden while still providing a clear idea of how the counter functions.

ctr is a module that represents an up/down counter, and it is possible to choose the actual physical implementation of the design from a wide variety of different styles of flops optimized for area, power and performance. They are usually compiled into libraries and will be available for us to select within EDA tools at a later stage in the design process.

Hardware Abstraction: Notice how you don't need to specify transistor-level details or gate connections. Verilog lets you describe *what* the circuit should do, and synthesis tools figure out *how* to build it in silicon.

How is Verilog different from software languages like C and Java ?

Verilog is a hardware description language (HDL) used to describe digital circuits and systems, while C and Java are software programming languages used to write code that runs on general-purpose computers. Here are some of the main differences between Verilog and programming languages like C and Java:

- **Purpose**: Verilog is used to describe digital circuits and systems, while C and Java are used to write software programs that run on computers.
- **Syntax**: Verilog has a different syntax than C and Java, as it is designed to describe the behavior of digital circuits rather than the execution of software instructions. For example, Verilog describes the properties of wires, registers, and logic gates, while C and Java define variables, functions, and control loops.
- **Execution**: Verilog is used to describe how digital circuits should behave, but it doesn't directly execute code. Instead, the Verilog code is compiled into a hardware configuration that can be implemented in a physical circuit or FPGA. C and Java code, on the other hand, is compiled into machine code that can be executed directly by a computer processor.
- **Testing and Verification**: Verilog is typically used to simulate the behavior of digital systems before they are physically implemented, while C and Java programs are usually tested and verified through software-based simulations or code reviews.
- **Nesting of Design**: In Verilog, the designs can be created as modules and can be reused which is not in the case of programming languages like C and Java where the code is written for a specific purpose.

Overall, Verilog is a specialized language designed specifically for digital circuit design and isn't used for general-purpose programming like C and Java. While there are some similarities in syntax and programming concepts between these languages, the primary focus and application of Verilog is on the design, simulation, and implementation of digital circuits and systems.

What may replace Verilog in the future ?

It's difficult to predict exactly what may replace Verilog in the future, but there are several emerging technologies and languages that may have an impact on the future of digital system design and verification.

One technology that may affect the future of digital system design is High-Level Synthesis (HLS), which is a technique for automatically generating hardware designs from high-level descriptions in

languages like C, C++, and SystemC. HLS allows designers to express their design intents and functionality at a higher level of abstraction, rather than specifying the details of logic gates and register transfers in Verilog or VHDL. This could enable more efficient and rapid design of digital systems, and allow designers to explore more design space in a shorter period of time.

Another technology that may impact the future of digital system design is machine learning and artificial intelligence (AI), which have the potential to significantly streamline the design and verification process of digital systems. For example, machine learning algorithms can be used to automatically optimize and generate hardware designs, reducing the need for manual design efforts.

There are also emerging HDLs that are trying to address some of the limitations of Verilog and VHDL, such as Chisel and MyHDL, which are based on more modern programming concepts and provide higher-level abstractions.

---

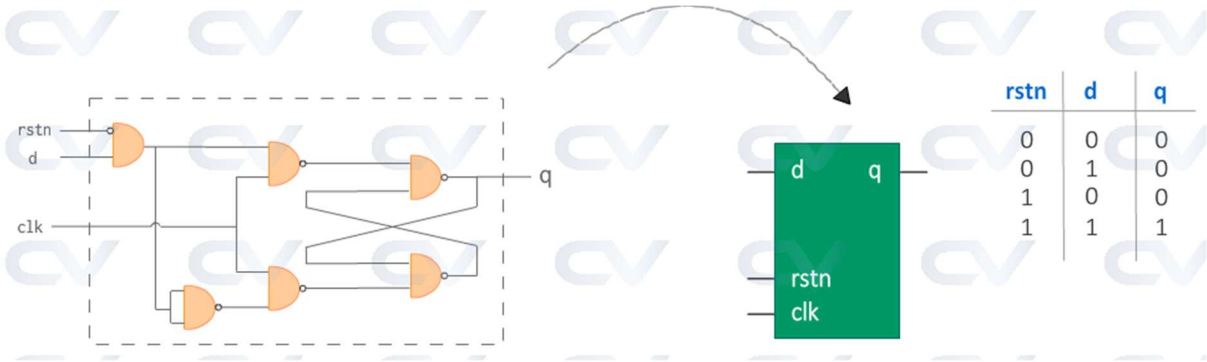
12 min read | Beginner

What You'll Learn

- What Verilog is and why hardware description languages revolutionized digital design
- How Verilog abstracts gate-level schematics into behavioral code
- The basic structure of a Verilog module including ports, signals, and behavioral blocks
- How testbenches verify hardware designs through simulation

From Gates to Hardware Description

A digital element such as a flip-flop can be represented with combinational gates like NAND and NOR. The functionality of a flip-flop is achieved by the connection of a certain set of gates in a particular manner. How the gates have to be connected is usually figured out by solving K-map from the truth table. The truth table is nothing but a table that tells us what inputs combine together to give what values of output. Shown in the image below is an electronic circuit that represents a D-flip flop and the corresponding truth table. The output **q** becomes 1 only when **rstn** and **d** are both having a value of 1.



What is a hardware schematic ?

A *hardware schematic* is a diagram that shows how the combinational gates should be connected to achieve a particular hardware functionality. In this case, it is the set of NAND gates connected like shown towards the left in the image above. However, if we know what values of inputs contribute to make the output have a value of 1, then we can essentially hide the internal details of the

connections and encapsulate it into a black-box. This block provides us with certain inputs and outputs that is similar to the hardware schematic made up of combinational gates.

What is a Hardware Description Language ?

It will be easier if we can describe how this block should behave and then let software tools convert that behavior into actual hardware schematic. The language that describes hardware functionality is called Verilog, and is classified as a *Hardware Description Language*.

What is meant by design functionality ?

Some typical behavioral requirements for a D-flip-flop are :

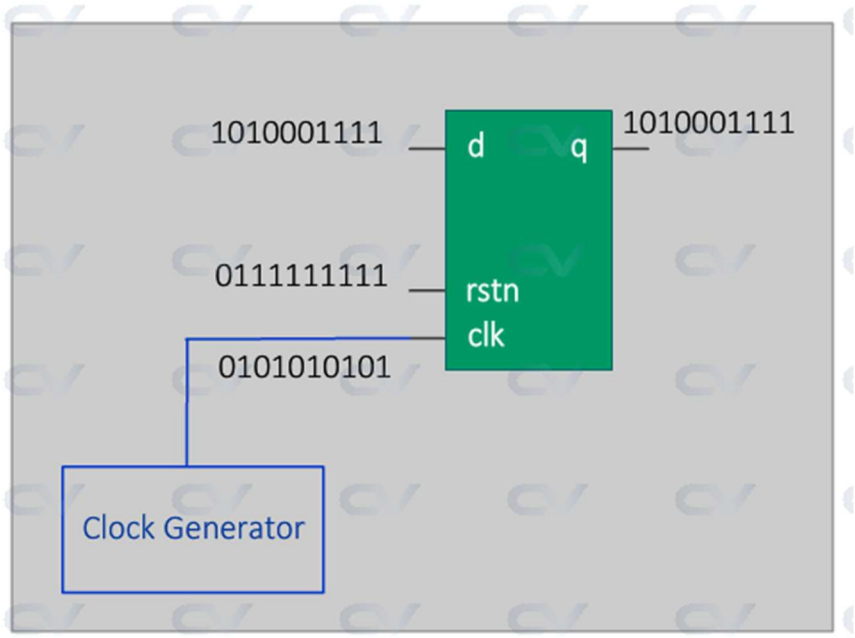
- clock should be an input to the flop
- if the active-low reset is 0, then the flop should reset
- if the active-low reset is 1, then the flop output 'q' should follow input 'd'
- output 'q' should get a new value only at the posedge of clock

An important question comes to mind : how do we know whether the behavior described in Verilog accurately reflects the intended behavior of the design ?

What is verification ?

This is checked by different methods and is collectively called as *verification*. The most common and widely practiced method of verification is circuit simulation. There are software tools to understand how a hardware described in Verilog should behave and provide various input stimuli to the design model. The output of the design is then checked against expected values to see if the design is functionally correct.

All simulations are performed by *EDA* (Electronic Design Automation) software tools and the Verilog design RTL is placed inside an entity called as *testbench*. Within the testbench, various tests provide different stimuli to the design. Such a testbench is shown in the image below.



Sections of Verilog code

All behavior code should be described within the keywords module and endmodule. Rest of the design code would mostly follow the given template.

Verilog sections template

1. Module definition and port list declaration
2. List of input and output ports
3. Declaration of other signals using allowed Verilog data types
4. Design may depend on other Verilog modules and hence their instances are created by module instantiations
5. The actual Verilog design for this module that describes its behavior

```
module [design_name] ( [port_list] );

        [list_of_input_ports]
        [list_of_output_ports]

        [declaration_of_other_signals]

        [other_module_instantiations_if_required]

        [behavioral_code_for_this_module]
endmodule
```

Example

The code shown below describes the behavior of a D type flip-flop. The first few lines declare a new module called **dff** and define the input and output ports. The only other signal used in this design is **q** and is declared next. Since this is a simple design, it does not depend on any other module and hence there are *no* module instantiations. The always block describes how the hardware should behave during certain events and hence is behavioral code.

// "dff" is the name of this module

```
module dff (    input   d,                      // Inputs to the design should start with "input"
                                                rstn,
                                                clk,
                                        output  q);             // Outputs of the design should start with
"output"

        reg q;                                                  // Declare a variable to store
output values

        always @ (posedge clk) begin    // This block is executed at the positive edge of clk 0->1
                if (!rstn)                              // At the posedge, if rstn is 0 then q should
get 0
                        q <= 0;
```

```
                else
                        q <= d;                         // At the posedge, if rstn is 1 then q should get d
        end
endmodule                                                       // End of module
```

Testbench code

The testbench is the Verilog container module that allows us to drive the design with different inputs and monitor its outputs for expected behavior. In the example shown below, we have *instantiated* the flop design illustrated above and connected it with testbench signals denoted by **tb_***. These testbench signals are then assigned certain values and are eventually driven as inputs to the design.

```
module tb;

        // 1. Declare input/output variables to drive to the design
        reg     tb_clk;
        reg     tb_d;
        reg     tb_rstn;
        wire    tb_q;

        // 2. Create an instance of the design
        // This is called design instantiation
        dff     dff0 (  .clk    (tb_clk),               // Connect clock input with TB signal
                        .d      (tb_d),         // Connect data input with TB signal
                        .rstn   (tb_rstn),              // Connect reset input with TB signal
                        .q      (tb_q));                // Connect output q with TB signal

        // 3. The following is an example of a stimulus
        // Here we drive the signals tb_* with certain values
        // Since these tb_* signals are connected to the design inputs,
        // the design will be driven with the values in tb_*
        initial begin
                tb_rstn <=      1'b0; // Start with reset asserted (active-low)
                tb_clk  <=      1'b0; // Initialize clock to 0
                tb_d    <=      1'b0; // Initialize data input to 0
        end
endmodule
```